

CSE 3204
Formal Language, Automata and Computability



Lecture 13
Pushdown Automata

Pushdown Automata

- Pushdown automata is an automaton for CFL.
- It is nondeterministic FA with ϵ -transitions + a stack
- It can remember infinite amount of information using a stack.

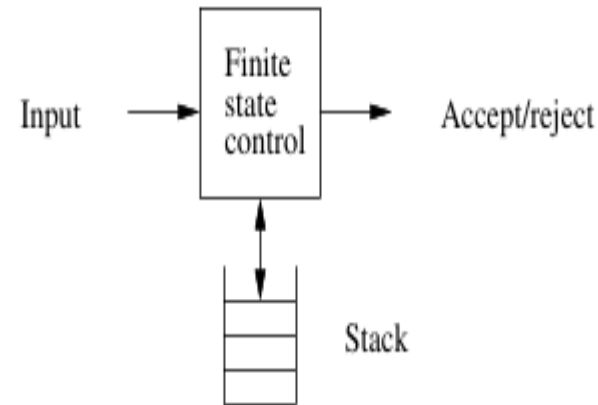


Figure 6.1: A pushdown automaton is essentially a finite automaton with a stack data structure

Pushdown Automata

- Observes the symbol at the top of the stack and makes transition based on its current state, the input symbol, and the symbol at the top of stack.
- Alternatively, it may make a spontaneous transition using ϵ as its input, instead of an input symbol.
- In one transition, the pushdown automata
 1. Consumes from the input the symbol that it uses in the transition. If ϵ is used for the input, then no input symbol is consumed.
 2. Goes to a new state, which may or may not be the same as the previous state.

Pushdown Automata

3. Replaces the symbol at the top of the stack by any string. The string could be ϵ , which corresponds to a pop of the stack. It could be the same symbol that appeared at the top of the stack previously; i.e., no change to the stack is made. It could also replace the top stack symbol by one other symbol, which in effect changes the top of the stack but does not push or pop it. Finally, the top stack symbol could be replaced by two or more symbols, which has the effect of (possibly) changing the top stack symbol, and then pushing one or more new symbols onto the stack.

Pushdown Automata

$$L_{ww^R} = \{ww^R \mid w \text{ is in } (0+1)^*\}$$

This language, often referred to as “ w - w -reversed,” is the even-length palindromes over alphabet $\{0, 1\}$. It is a CFL, generated by the grammar of Fig. 5.1, with the productions $P \rightarrow 0$ and $P \rightarrow 1$ omitted.

1. Start in a state q_0 that represents a “guess” that we have not yet seen the middle; i.e., we have not seen the end of the string w that is to be followed by its own reverse. While in state q_0 , we read symbols and store them on the stack, by pushing a copy of each input symbol onto the stack, in turn.
2. At any time, we may guess that we have seen the middle, i.e., the end of w . At this time, w will be on the stack, with the right end of w at the top and the left end at the bottom. We signify this choice by spontaneously going to state q_1 . Since the automaton is nondeterministic, we actually make both guesses: we guess we have seen the end of w , but we also stay in state q_0 and continue to read inputs and store them on the stack.
3. Once in state q_1 , we compare input symbols with the symbol at the top of the stack. If they match, we consume the input symbol, pop the stack, and proceed. If they do not match, we have guessed wrong; our guessed w was not followed by w^R . This branch dies, although other branches of the nondeterministic automaton may survive and eventually lead to acceptance.
4. If we empty the stack, then we have indeed seen some input w followed by w^R . We accept the input that was read up to this point.

Formal Definition of Pushdown Automata

$$P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

The components have the following meanings:

Q : A finite set of *states*, like the states of a finite automaton.

Σ : A finite set of *input symbols*, also analogous to the corresponding component of a finite automaton.

Γ : A finite *stack alphabet*. This component, which has no finite-automaton analog, is the set of symbols that we are allowed to push onto the stack.

δ : The *transition function*. As for a finite automaton, δ governs the behavior of the automaton. Formally, δ takes as argument a triple $\delta(q, a, X)$, where:

1. q is a state in Q .
2. a is either an input symbol in Σ or $a = \epsilon$, the empty string, which is assumed not to be an input symbol.
3. X is a stack symbol, that is, a member of Γ .

The output of δ is a finite set of pairs (p, γ) , where p is the new state, and γ is the string of stack symbols that replaces X at the top of the stack. For instance, if $\gamma = \epsilon$, then the stack is popped, if $\gamma = X$, then the stack is unchanged, and if $\gamma = YZ$, then X is replaced by Z , and Y is pushed onto the stack.

q_0 : The *start state*. The PDA is in this state before making any transitions.

Z_0 : The *start symbol*. Initially, the PDA's stack consists of one instance of this symbol, and nothing else.

F : The set of *accepting states*, or *final states*.

Design of a PDA for $L_{ww^R} = \{ww^R \mid w \text{ is in } (0+1)^*\}$

$$P = (\{q_0, q_1, q_2\}, \{0, 1\}, \{0, 1, Z_0\}, \delta, q_0, Z_0, \{q_2\})$$

$$5. \delta(q_1, \epsilon, Z_0) = \{(q_2, Z_0)\}$$

$$1. \delta(q_0, 0, Z_0) = \{(q_0, 0Z_0)\} \text{ and } \delta(q_0, 1, Z_0) = \{(q_0, 1Z_0)\}.$$

$$2. \delta(q_0, 0, 0) = \{(q_0, 00)\}, \delta(q_0, 0, 1) = \{(q_0, 01)\}, \delta(q_0, 1, 0) = \{(q_0, 10)\},$$

$$\delta(q_0, 1, 1) = \{(q_0, 11)\}.$$

$$3. \delta(q_0, \epsilon, Z_0) = \{(q_1, Z_0)\}, \delta(q_0, \epsilon, 0) = \{(q_1, 0)\}, \text{ and } \delta(q_0, \epsilon, 1) = \{(q_1, 1)\}$$

$$4. \delta(q_1, 0, 0) = \{(q_1, \epsilon)\}, \text{ and } \delta(q_1, 1, 1) = \{(q_1, \epsilon)\}$$

Graphical Notation for a PDA

- a) The nodes correspond to the states of the PDA.
- b) An arrow labeled *Start* indicates the start state, and doubly circled states are accepting, as for finite automata.
- c) The arcs correspond to transitions of the PDA in the following sense. An arc labeled $a, X/\alpha$ from state q to state p means that $\delta(q, a, X)$ contains the pair (p, α) , perhaps among other pairs. That is, the arc label tells what input is used, and also gives the old and new tops of the stack.

The only thing that the diagram does not tell us is which stack symbol is the start symbol. Conventionally, it is Z_0 , unless we indicate otherwise.

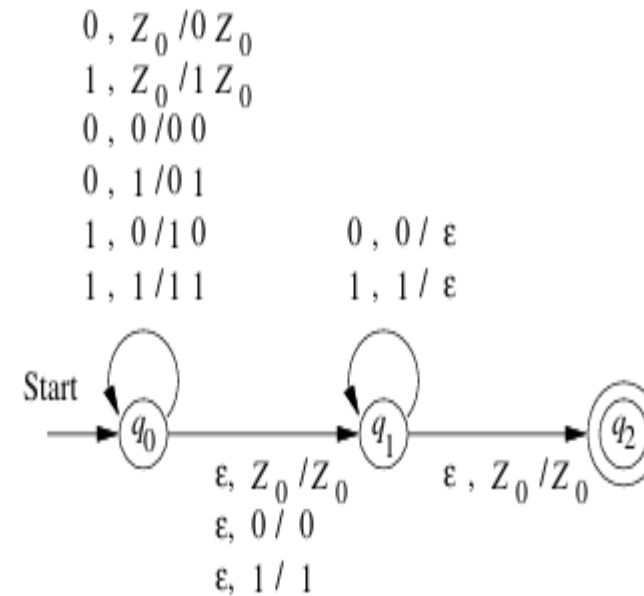


Figure 6.2: Representing a PDA as a generalized transition diagram

Instantaneous Descriptions of a PDA

Thus, we shall represent the configuration of a PDA by a triple (q, w, γ) , where

1. q is the state,
2. w is the remaining input, and
3. γ is the stack contents.

Conventionally, we show the top of the stack at the left end of γ and the bottom at the right end. Such a triple is called an *instantaneous description*, or ID, of the pushdown automaton.

OR MANY MOVES OF A PDA.

Let $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ be a PDA. Define $\vdash_P$, or just $\vdash$ when P is understood, as follows. Suppose $\delta(q, a, X)$ contains (p, α) . Then for all strings w in Σ^* and β in Γ^* :

$$(q, aw, X\beta) \vdash (p, w, \alpha\beta)$$

We also use the symbol $\vdash_P^*$, or $\vdash^*$ when the PDA P is understood, to represent zero or more moves of the PDA. That is:

BASIS: $I \vdash_P^* I$ for any ID I .

INDUCTION: $I \vdash_P^* J$ if there exists some ID K such that $I \vdash_P K$ and $K \vdash_P^* J$.

That is, $I \vdash_P^* J$ if there is a sequence of ID's $K_1, K_2, \dots, K_n$ such that $I = K_1$, $J = K_n$, and for all $i = 1, 2, \dots, n-1$, we have $K_i \vdash_P K_{i+1}$.

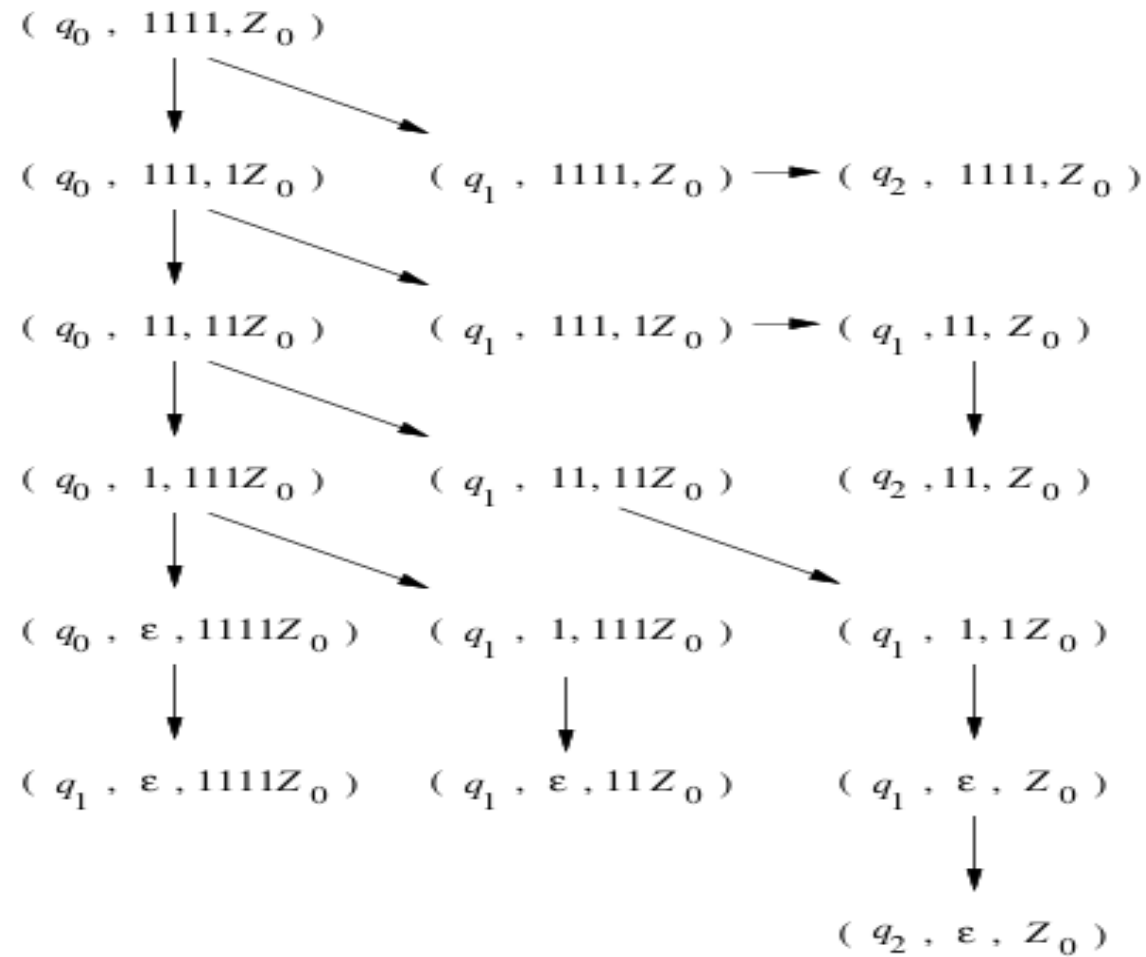


Figure 6.3: ID's of the PDA of Example 6.2 on input 1111

Three important principles about IDs and their transitions

1. If a sequence of ID's (*computation*) is legal for a PDA P , then the computation formed by adding the same additional input string to the end of the input (second component) in each ID is also legal.
2. If a computation is legal for a PDA P , then the computation formed by adding the same additional stack symbols below the stack in each ID is also legal.
3. If a computation is legal for a PDA P , and some tail of the input is not consumed, then we can remove this tail from the input in each ID, and the resulting computation will still be legal.

Theorem 6.5: If $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ is a PDA, and $(q, x, \alpha) \vdash_P^* (p, y, \beta)$, then for any strings w in Σ^* and γ in Γ^* , it is also true that

$$(q, xw, \alpha\gamma) \vdash_P^* (p, yw, \beta\gamma)$$

Note that if $\gamma = \epsilon$, then we have a formal statement of principle (1) above, and if $w = \epsilon$, then we have the second principle.

Three important principles about IDs and their transitions

Theorem 6.6: If $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ is a PDA, and

$$(q, xw, \alpha) \vdash_P^* (p, yw, \beta)$$

then it is also true that $(q, x, \alpha) \vdash_P^* (p, y, \beta)$. $\square$

- Section 6.1 of Chapter 6